L'Architettura della Memoria Virtuale

Cos'è la Memoria Virtuale?

L'evoluzione del software moderno ha portato al fenomeno del **bloatware**, ovvero programmi così pesanti che la RAM fisica spesso non è sufficiente a gestirli tutti contemporaneamente.
In questo scenario, la memoria virtuale non è un semplice "trucco" per guadagnare spazio, ma l'astrazione fondamentale (ovvero un'intermediario) che separa *ciò che il programma vede* dalla *reale struttura fisica della RAM.*

Ogni processo vive in un proprio "spazio virtuale" diviso in pagine.
Invece di caricare l'intero programma in memoria, il sistema mantiene nella RAM fisica solo le pagine necessarie in quel momento. Quando il processo richiede un indirizzo virtuale specifico, l'astrazione interviene per "tradurre" istantaneamente quell'indirizzo nella cella fisica corrispondente.

La Paginazione

Il meccanismo principale che permette tutto questo (ovvero che permette la gestione della memoria) si chiama **paginazione**. Funziona così:

- Lo spazio virtuale viene diviso in pezzi fissi chiamati **pagine**.
- La memoria fisica viene divisa in blocchi identici chiamati **page frames**.

Tuttavia, la memoria fisica è limitata: non tutte le pagine virtuali possono essere mappate contemporaneamente nei frame.
Quando il processore cerca un'informazione, interroga la MMU (l'unità che gestisce la memoria). La MMU cerca, in primis, se l'informazione è presente nel TLB (Translation Lookaside Buffer) ovvero una memoria cache.

- Se quest'informazione **è presente** allora si avrà un TLB HIT, ovvero l'indirizzo è presente e lo restituisco.
- Se quest'informazione **non è presente** allora si avrà un TLB MISS, ovvero l'indirizzo non è presente ed esegue una normale ricerca sulla tabella della pagine.

Se la MMU scopre che la pagina richiesta non è presente in RAM, genera un'eccezione chiamata Page Fault.

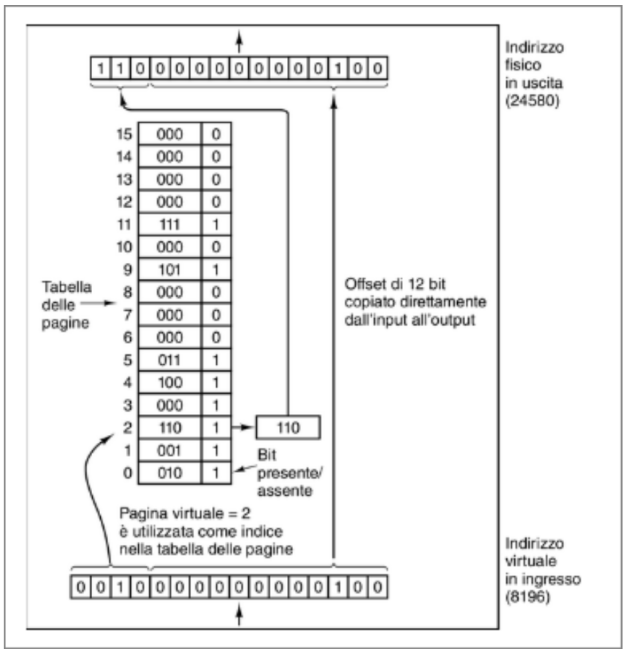


Figura 1: MMU

Tabella delle Pagine

Una Tabella della pagine è composta da uno Spazio di Indirizzi Virtuali, che vengono tradotti in Indirizzi Fisici. Questa traduzione viene fatta dalla MMU(l'unità che gestisce la memoria). Un indirizzo virtuale si trova nella pagine, e queste vengono mappate attraverso l'MMU nella memoria fisica (RAM).

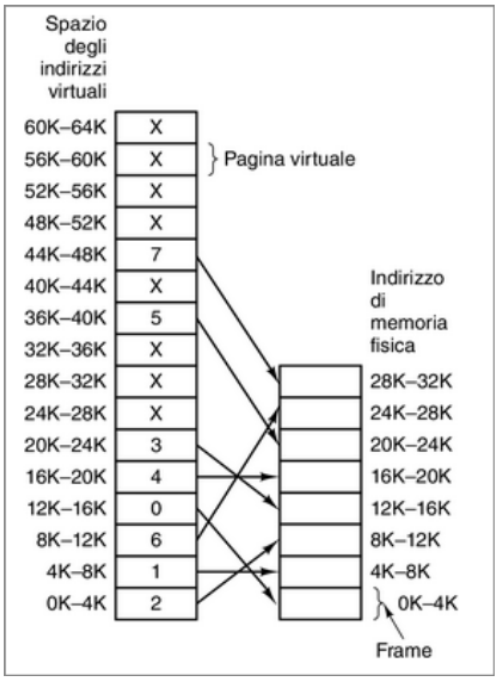


Figura 2: Esempio di Tabella della Pagine con 16 pagine e 8 frame

Velocità e Prestazioni: Il TLB

Consultare la tabella delle pagine in RAM per ogni singola operazione sarebbe troppo lento e raddoppierebbe i tempi di accesso. Per risolvere il problema, usiamo il **TLB (Translation Lookaside Buffer)**.

Il TLB è una cache hardware velocissima integrata nel processore che tiene a mente le traduzioni usate più di recente. Grazie al fatto che i programmi tendono a riusare spesso gli stessi dati (**principio di località**), quasi tutte le traduzioni avvengono istantaneamente nel TLB, mantenendo il sistema fluido.

Gestione del Page Fault

Può accadere che la RAM è piena, ma grazie alla gestione della memoria virtuale non è necessario che tutto un programma sia in RAM per farlo girare; il sistema carica solo le pagine che servono davvero.
Se un processo prova a usare una pagina che non è ancora stata mappata in RAM (lo capiamo da un **bit di presenza/assenza** nella tabella), la MMU scatena un'eccezione chiamata **page fault**.

A questo punto interviene il Sistema Operativo:

1. Sospende momentaneamente il processo.
2. Cerca la pagina mancante sul disco (memoria secondaria).
3. Trova un frame libero in RAM (se non c'è spazio, usa algoritmi di sostituzione delle pagine per liberarne uno).
4. Carica i dati nel frame e aggiorna la tabella delle pagine.
5. Fa ripartire l'istruzione interrotta in modo totalmente trasparente per l'utente.